



Network



Summary On Lec 4

❖ هنتكلم ف المحاضرة دي عن ال Transport Layer :-

➔ provide **logical communication** between application processes running on different hosts

- الابلكيشن ال عايز بيعت مسدج ببيعتها لل Transport Layer ال عنده بس هيقسمها ل Segments وهي بتبقي مسئولة عن انها تبعتها لل Transport Layer الموجوده ف الناحيه الثانيه وف الاخر تقول للابلكيشن ايه ال حصل

➤ Transport vs. network layer services and protocols :-

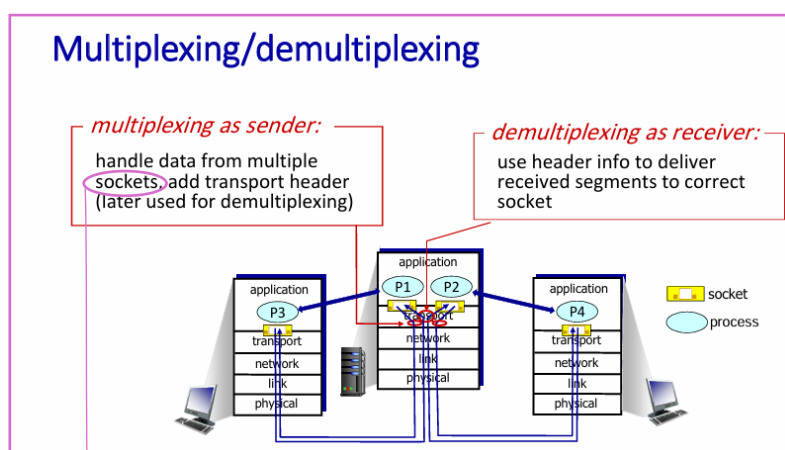
▪ **transport layer:**
communication between **processes**

- relies on, enhances, network layer services

▪ **network layer:**
communication between **hosts**

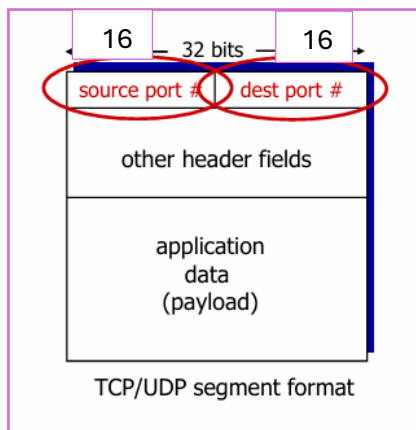
- يبقى ال Transport Layer مسئولة عن ال Logical Communication بين ال Processes ال لازم تبقي موجوده عند ال Sender و ال Receiver End Systems وبالتالي البرنامج ال هنا بيعتمد عليها علشان يكلم البرنامج ال هناك
- اما ال Network Layer بنتكلم عن جوا النيتورك ومسئوليتها عن المسارات ال هتاخذ فيها الداتا

➔ Multiplexing/Demultiplexing :-



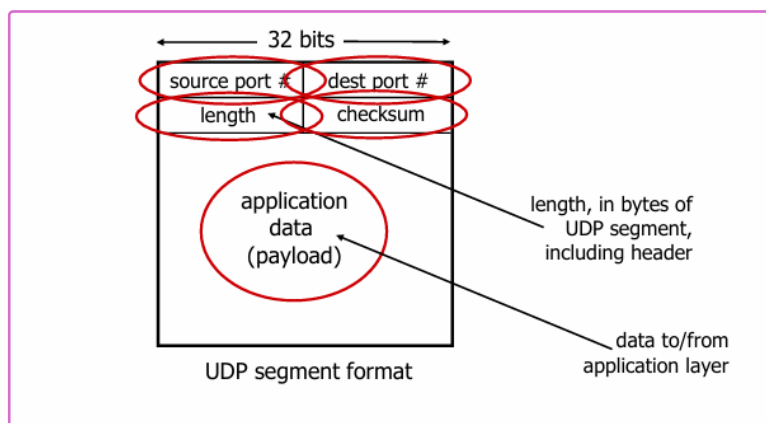
اشبه بال APIs بين الابلكيشن وال Transport Layer

- مين دول بقي؟
- **Multiplexing** هنا الجهاز اللي بيعت (زي الكمبيوتر الأول في الصورة) بياخد بيانات من برامج أو عمليات مختلفة زي P3 و P4 ويضيف ليها معلومات في ال Header بتاعها ، علشان يعرف يفرق بينهم بعدين لما يوصلوا لجهاز المستقبل.
- **Demultiplexing** هنا الجهاز اللي بيستقبل (زي الكمبيوتر الثاني)، بيستخدم المعلومات اللي في ال header علشان يفرق بين البيانات اللي جاية من برامج مختلفة وبيعتها للبرنامج أو العملية الصح زي P1 و P2



- ال بيحصل ان لما تيجي المسدج ال هتتبعث يعني ال Transport Layer هتتبعث الداتا دي فبتتخزن ف (Payload) لكن لما تروح لل Network Layer بيكون قبلها Header

- TCP socket identified by 4-tuple:
 - Source IP address
 - Source port number
 - Dest IP address
 - Dest port number



- ايه ال CheckSum دا؟
عبارة عن حسبه بتتسبب بفتح في ال Header بتاع ال UDP علشان المسدج لما توصل عند ال Receiver ال يقدر يضمن ان هي محصلهاش ايرورز علي النيتورك وبالتالي المستلم يقدر يحسب ال CheckSum هو كمان وبناءا عليه يقدر يقول ال Segment دي سليمة ولا لا لو سليمة هيبعتها للابليكيشن لو مش سليمة هيفكسلها ولا كأنها موجوده

--> طيب الحسبه دي بتتم ازاى؟

- ال Sender بيمسك ال Segment وبيتيدي يقسمها 2-Bytes وبيتيدي ياخذ البلوكات ال قسمها دي ويجمعها علي بعض بطريقة ما هنشوفها الوقتي والناجح بيحطه ف ال CheckSum بقي

- ال Receiver لما بيستلم المسدج بيعمل نفس الحكاية

example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

- زي ما اتفقنا انه بيقطع المسدج ل 2-Bytes = 16-Bit ف بياخد كل واحد مع ال بعده وجمعهم مع بعض جمع باينري عادي طيب لو لقيت معايا Carry هأخذ ال Carry دا واروح اجمعه علي اول Bit تاني لحد ما الناتج يطلع مطبوظ ف اخذ الناتج دا واقلبه يعني اخلي ال 0 بيفي 1 وال 1 بيفي 0 ويبقى هو دا ال CheckSum

- لما ال Receiver يستلم ايه ال هيصصل؟

On the receiver

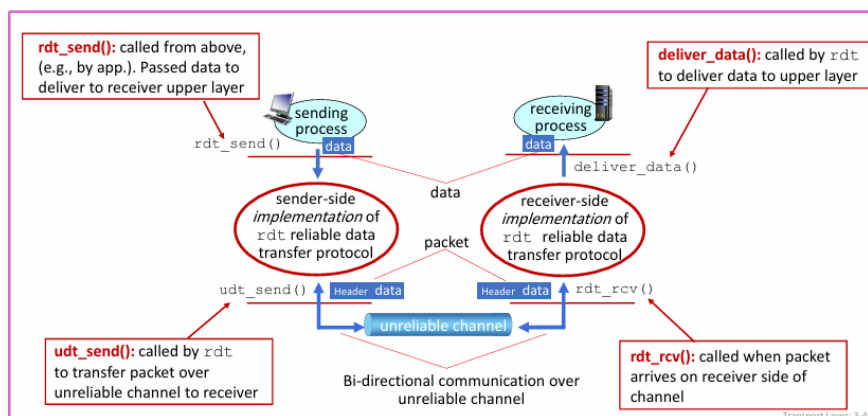
add the three 16-bit integers (including checksum)

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1
	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

- هيستقبل المسدج + ال CheckSum ويعمل نفس عملية الجمع ولو طلع Carry ف الآخر يجمعها علي اول Bit برده بعدين ياخذ النتيجة النتيجة دي بقي لو طلعت كلها 1 بيفي المسدج سليمة لو طلع اي 0 فيها بيفي المسدج فيها ايرور

- يعني ايه **Reliable Data Transfer**؟

دا معناه ان ال بيبعت داتا يقدر يعرف الداتا بتاعته وصلت ولا لا ولو موصلتش يقدر بيعتها كذا مرة لحد ما يضمن ان هي وصلت وبالتالي ف نهاية عملية الارسال بقي عارف الداتا بتاعته وصلت وسليمة ولا موصلتش خالص ولو دا حصل هبقي عارف موصلتش ليه



- ببدا ابعت الداتا ال rdt هياخد الداتا زي ما هي ويحطها ف شكل Segment/Packet ويضيف عليها ال Headers بتاعته ويحصل عليها شوية Processing ونتيجة ال Processing دي هيتحط

حاجه علي التشاغل ويحطها علي ال unreliable channel هتوصل الناحية التانية ال Receiver هيستقبل ال جايه هيشيل ال Headers وياخد الداتا ويعمل العملية العكسية و يديها للابلكيشن ومش هحتاج اضيف ال CheckSum ف الحالة دي



لا تنسو الدعاء لآخواننا في
فلسطين و غزة و العراق و
لبنان و سوريا و السودان.
و تذكر دائما المقاطعة ولا
تتخلي عن مبادئك و انت بتجاهد
معاهم بمقاطعتك.